



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorio de docencia

Laboratorio de Computación Salas A y B

Profesor(a):

M.C. René Adrián Dávila Pérez

Asignatura:

Programación orientada a objetos

Grupo:

07

No de práctica(s):

05

Integrante(s):

424045919 | 323087067 | 426060033 | 426059961 | 323297930

No de lista o brigada:

1

Semestre:

2027-1

Fecha de entrega:

02 de octubre de 2026

Observaciones:

Calificación:

Índice

1. Introducción	2
1.1. Planteamiento del Problema	2
1.2. Motivación	2
2. Objetivos	2
2.1. Objetivo General	2
2.2. Objetivos Específicos	2
3. Marco Teórico	3
3.1. Abstracción y Encapsulamiento en POO	3
3.2. Niveles de Visibilidad y Modificadores de Acceso	3
3.3. Métodos Modificadores (Getters y Setters) y Sobrescritura (@Override)	3
4. Desarrollo	4
4.1. Actividad 1: Consulta Técnica con IA Generativa sobre Constructores y Modificadores	4
4.2. Actividad 2: Implementación de la Clase Empleado y Validación de Estado	4
4.3. Actividad 3: Implementación de la Clase Fecha y Formato de Salida	4
4.4. Diagramas de Clases UML	5
4.5. Casos de Prueba de Entrada y Salida	5
5. Resultados	5
5.1. Consultas a IAs Generativas por la Brigada	5
5.2. Evidencias de Ejecución en Terminal	7
6. Conclusiones	7
7. Referencias	8

1. Introducción

1.1. Planteamiento del Problema

En el desarrollo de software bajo el paradigma de la Programación Orientada a Objetos, uno de los desafíos centrales consiste en modelar correctamente los atributos y comportamientos de las entidades del mundo real resguardando la integridad de sus datos. La falta de mecanismos de protección en el estado interno de un objeto permite que agentes externos alteren sus variables de forma directa, ocasionando inconsistencias lógicas en el sistema. Para abordar esta problemática, en esta práctica se plantea la resolución de tres actividades enfocadas en la abstracción y el encapsulamiento:

1. **Análisis Conceptual sobre Inicialización de Objetos:** Analizar y evaluar con el apoyo de herramientas de Inteligencia Artificial la conveniencia técnica de invocar métodos modificadores (setters) dentro de los constructores de una clase, redactando una justificación conceptual.
2. **Sistema de Control de Salarios (Empleado / MainEmpleado):** Diseñar una clase para representar empleados con atributos de nombre, apellido y salario mensual. El programa debe incluir una regla de negocio estricta: si el salario asignado mediante el constructor o el método modificador no es un valor positivo, dicho valor no debe establecerse en el atributo. Asimismo, mediante una clase probadora se deben instanciar dos objetos de tipo Empleado, calcular y mostrar su salario anual, aplicar un incremento del 10 % y desplegar nuevamente el salario anual actualizado.
3. **Gestión de Entidades de Tiempo (Fecha / MainFecha):** Implementar una clase que represente una fecha mediante tres atributos enteros (mes, día y año). La clase debe incluir un constructor que asuma valores válidos, métodos modificadores para cada atributo y el método mostrarFecha(), el cual despliega el día, mes y año delimitados por barras diagonales (/). La implementación se prueba a través de una clase principal independiente.

1.2. Motivación

La correcta aplicación de la abstracción y el encapsulamiento permite aislar los detalles de implementación de una clase respecto a su interfaz pública, garantizando la seguridad en el manejo de memoria y reduciendo el acoplamiento entre los componentes del software. Comprender el funcionamiento de los modificadores de acceso (private, public, protected, paquete) y la responsabilidad de los métodos get y set es esencial para prevenir que un objeto alcance estados inconsistentes, tales como salarios negativos o formatos de fecha erróneos. Además, evaluar mediante diversas IAs las buenas prácticas en la construcción de objetos fomenta un criterio técnico crítico sobre el diseño orientado a objetos en la ingeniería de software.

2. Objetivos

2.1. Objetivo General

Aplicar el concepto de abstracción para el diseño de clases que integran una solución, utilizando el encapsulamiento para proteger la información y ocultar la implementación.

2.2. Objetivos Específicos

- Seleccionar los atributos y comportamientos relevantes de una entidad mediante abstracción para modelar las clases Empleado y Fecha.
- Implementar modificadores de acceso privados (private) en los atributos de clase y proveer métodos públicos de acceso (getters y setters).
- Integrar lógica de validación dentro de los métodos modificadores para rechazar valores inválidos, garantizando la consistencia de los datos en memoria.

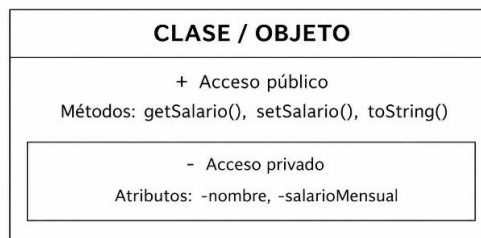
- Consultar y comparar las respuestas de cinco herramientas de IA Generativa respecto al uso de métodos modificadores dentro de constructores.
- Desarrollar clases probadoras para verificar la manipulación indirecta de los datos, el cálculo de incrementos porcentuales y la salida formateada de cadenas.

3. Marco Teórico

3.1. Abstracción y Encapsulamiento en POO

La abstracción es el proceso mediante el cual se separan los datos e interacciones relevantes de una entidad ignorando aquellos aspectos irrelevantes para el problema que se busca resolver[1,2]. Este principio permite enfocar la atención en el nivel alto del problema respondiendo a la pregunta “¿qué hace el objeto?”[1,2].

Por otro lado, el encapsulamiento envuelve los datos dentro de una capa protectora dentro de la clase[1,4]. Esto significa que los datos internos permanecen ocultos y protegidos ante accesos o modificaciones arbitrarias desde el exterior[4,5]. Para interactuar con dichos atributos, se proveen mecanismos específicos llamados métodos modificadores (getters y setters)[5,6].



3.2. Niveles de Visibilidad y Modificadores de Acceso

Java ofrece cuatro niveles de visibilidad aplicables tanto a atributos como a métodos para regular la accesibilidad entre las clases del sistema[3]:

- **Privado (private):** Modificador más restrictivo; limita el acceso únicamente a los miembros de la misma clase[3,5].
- **Protegido (protected):** Oculta los datos al exterior del paquete, pero permite el acceso a clases hijas mediante herencia[3].
- **Paquete (Default):** Nivel de acceso por defecto cuando no se especifica modificador; restringe la visibilidad a las clases dentro del mismo paquete[3].
- **Público (public):** Expone el miembro para que pueda ser accedido desde cualquier clase del proyecto[3].

3.3. Métodos Modificadores (Getters y Setters) y Sobrescritura (@Override)

Los métodos get no reciben parámetros y retornan el valor almacenado en un atributo privado. Los métodos set reciben un parámetro con el nuevo dato y permiten incorporar validaciones lógicas antes de realizar la asignación[5,6].

Adicionalmente, el método toString() pertenece a la clase base Object y se encarga de convertir el estado del objeto a una representación en cadena de texto[2]. Al utilizar la anotación @Override, se redefine dicho método para personalizar el formato de salida, aplicando un esquema de polimorfismo en tiempo de ejecución[2].

4. Desarrollo

4.1. Actividad 1: Consulta Técnica con IA Generativa sobre Constructores y Modificadores

Para el primer ejercicio, cada integrante de la brigada utilizó una herramienta de Inteligencia Artificial distinta para resolver la interrogante planteada en la sesión: “¿Es conveniente usar métodos modificadores en los constructores? Justificación de al menos media cuartilla”.

El análisis consensuado indica que delegar las asignaciones del constructor hacia los setters favorece el principio de reutilización de código al concentrar las validaciones en un solo punto. No obstante, existe un riesgo técnico de sobrepaso de métodos en escenarios de herencia: si un constructor llama a un setter público que es sobrescrito por una subclase, el método de la subclase se ejecutará antes de que la clase base concluya su proceso de instanciación, lo que puede provocar inconsistencias o errores de puntero nulo. Por ello, la buena práctica dicta emplear setters en constructores únicamente si son declarados como `private` o `final`.

4.2. Actividad 2: Implementación de la Clase Empleado y Validación de Estado

Se implementó la clase de dominio `Empleado.java` aplicando encapsulamiento estricto sobre sus tres atributos: `nombre`, `apellido` y `salarioMensual`.

1. **Estructura del Constructor:** Recibe los tres parámetros iniciales. Se efectúa la asignación de nombre y apellido, mientras que para el salario mensual se valida la condición. Si el argumento ingresado es mayor a cero, se asigna al atributo; en caso contrario, el valor no se establece (manteniendo el valor por defecto de 0.0).
2. **Lógica de Métodos Modificadores:** El método `setSalarioMensual` contiene una estructura condicional `if` (`salarioMensual > 0`). Si la condición es verdadera, actualiza la variable de instancia; si es falsa, la instrucción se omite para proteger la integridad del objeto.
3. **Ejecución en Clase de Prueba (MainEmpleado.java):** Se instancian dos objetos `Empleado` con salarios válidos. Se calcula el salario anual multiplicando el monto mensual por 12 y se imprime en pantalla. Posteriormente, se calcula un incremento del 10 % en el salario mensual mediante el método modificador y se vuelve a desplegar el salario anual acumulado.

4.3. Actividad 3: Implementación de la Clase Fecha y Formato de Salida

Se diseñó la clase `Fecha.java` estructurando sus tres atributos privados de tipo entero: `mes`, `dia` y `anio`.

1. **Construcción e Inicialización:** El constructor toma los tres enteros recibidos y los asigna directamente a sus campos correspondientes asumiendo datos válidos.
2. **Método de Despliegue (mostrarFecha):** Se implementó una función que concatena las variables numéricas separadas por el carácter de barra diagonal (/), desplegando en consola la secuencia en formato `mes/dia/anio`.
3. **Clase Principal (MainFecha.java):** Instancia un objeto `Fecha`, demuestra el uso de sus métodos `set` y `get` y ejecuta `mostrarFecha()` para validar la salida limpia en terminal.

4.4. Diagramas de Clases UML



Figura 1. Diagramas de clases UML para los programas MainFecha/Fecha y MainEmpleado/Empleado.

4.5. Casos de Prueba de Entrada y Salida

- **Prueba 1 (Validación de Salario Inválido en Empleado):**

- *Entrada:* Se intenta crear un Empleado con salario de -5000.0.
- *Resultado:* El programa detecta un valor no positivo, no establece la cantidad y conserva el salario en 0.0.

- **Prueba 2 (Cálculo de Incremento en Empleado):**

- *Entrada:* Empleado 1 con salario de 10000.0; Empleado 2 con salario de 20000.0.
- *Resultado:* Salarios anuales iniciales de 120000.0 y 240000.0 respectivamente. Tras otorgar el aumento del 10 %, los nuevos salarios anuales desplegados son 132000.0 y 264000.0.

- **Prueba 3 (Formato en Fecha):**

- *Entrada:* mes = 9, dia = 23, anio = 2026.
- *Resultado:* Invocación de `mostrarFecha()` imprime la cadena "9/23/2026" en consola.

5. Resultados

5.1. Consultas a IAs Generativas por la Brigada

A continuación se recopilan las consultas individuales realizadas por los cinco integrantes de la brigada a diferentes modelos de IA para abordar el Ejercicio 1:

[Integrante 1: Bazán Pantaleón Rosa María | Herramienta: ChatGPT]

PROMPT:

“¿Es conveniente usar métodos modificadores (setters) dentro de los constructores en Java? Justifica técnicamente en media cuartilla.”

RESPUESTA:

“Es conveniente si se busca centralizar las reglas de validación y evitar duplicar código entre el constructor y los setters. Sin embargo, en jerarquías de herencia representa un riesgo: si el setter es público o protegido y no es 'final', una clase derivada podría sobrescribirlo y ejecutarse sobre un objeto no inicializado completamente.”

[Integrante 2: García Rivera Gabriel Eduardo | Herramienta: Claude]

PROMPT:

“Analiza las ventajas y desventajas de invocar setters dentro de un constructor en POO Java.”

RESPUESTA:

“La ventaja principal es la adherencia al principio DRY al no repetir la lógica de validación de campos. La desventaja crítica es la invocación de métodos polimórficos desde constructores. Si el setter es modificado por una subclase, la ejecución puede acceder a atributos no inicializados del hijo, lanzando excepciones en tiempo de ejecución.”

[Integrante 3: Padrón López María Fernanda Corina | Herramienta: Gemini]

PROMPT:

“¿Por qué se considera riesgoso llamar a métodos modificadores dentro del constructor en Java? Proporciona una justificación académica.”

RESPUESTA:

“El riesgo proviene del estado de inicialización parcial del objeto. Llamar a setters sobrescribibles viola el orden seguro de construcción. Como buena práctica de diseño, se recomienda asignar los campos directamente en el constructor o invocar únicamente métodos privados o finales.”

[Integrante 4: Toledo Valencia Aimée Guadalupe | Herramienta: Copilot]

PROMPT:

“¿Cuándo es correcto utilizar setters dentro de los constructores según las buenas prácticas de Java?”

RESPUESTA:

“Es una práctica correcta solo si los setters están marcados como privados o finales. De este modo, se garantiza que la validación se ejecute de forma uniforme sin permitir que subclases alteren el comportamiento de construcción.”

[Integrante 5: Vitla Herrera Cesar Antonio | Herramienta: DeepSeek]

PROMPT:

“Justifica conceptualmente la conveniencia o inconveniencia de usar métodos modificadores en el constructor de una clase.”

RESPUESTA:

“Desde el enfoque de encapsulamiento, usar setters en constructores asegura que el objeto nazca en un estado válido. No obstante, invocar métodos modificadores abiertos en el constructor vulnera la seguridad del objeto en escenarios complejos. La alternativa estándar es emplear funciones privadas de verificación.”

5.2. Evidencias de Ejecución en Terminal

```
PS C:\Users\Gabo\Desktop\Programacion\P00\P00-E1-T-\P5> javac MainEmpleado.java
PS C:\Users\Gabo\Desktop\Programacion\P00\P00-E1-T-\P5> java MainEmpleado
REGISTRO DEL EMPLEADO 1
Ingrese el nombre: Gabriel
Ingrese el apellido: Rivera
Ingrese el salario mensual (debe ser mayor o igual a 0): -300
Error: El salario no puede ser negativo. Intente nuevamente.
Ingrese el salario mensual (debe ser mayor o igual a 0): 1000

REGISTRO DEL EMPLEADO 2
Ingrese el nombre: Fernanda
Ingrese el apellido: Padron
Ingrese el salario mensual (debe ser mayor o igual a 0): 5000

SALARIOS ANUALES INICIALES
Gabriel Rivera: 12000.0
Fernanda Padron: 60000.0

SALARIOS TRAS AUMENTO DEL 10%
Gabriel Rivera
Mensual: 1100.0
Anual: 13200.0

Fernanda Padron
Mensual: 5500.0
Anual: 66000.0
PS C:\Users\Gabo\Desktop\Programacion\P00\P00-E1-T-\P5>

PS C:\Users\Gabo\Desktop\Programacion\P00\P00-E1-T-\P5> javac MainFecha.java
PS C:\Users\Gabo\Desktop\Programacion\P00\P00-E1-T-\P5> java MainFecha
REGISTRO DE FECHA
Ingrese el mes (1-12): 10
Ingrese el día (1-31): 2
Ingrese el año: 2026

La fecha ingresada es: 10/2/2026
PS C:\Users\Gabo\Desktop\Programacion\P00\P00-E1-T-\P5>
```

Figura 3. Ejecución en terminal del programa.

6. Conclusiones

Los objetivos establecidos para esta práctica se cumplieron en su totalidad al aplicar los principios de abstracción y encapsulamiento en la construcción de soluciones orientadas a objetos en Java. La implementación de la clase Empleado demostró la importancia de ocultar los datos mediante el modificador `private` y condicionar su mutación a través de métodos modificadores. Esta estructura garantizó que la regla de negocio —rechazar salarios no positivos— se cumpliera de manera rigurosa tanto en la fase de instanciación como en modificaciones posteriores, evitando la presencia de estados inconsistentes en la memoria del programa.

Por otro lado, la clase Fecha confirmó la utilidad de la abstracción al aislar los datos numéricos de una entidad de tiempo y exponer únicamente un método público de despliegue que formatea la salida sin alterar los campos internos. Asimismo, la simulación realizada sobre el uso de modificadores en constructores permitió identificar las implicaciones técnicas del polimorfismo dinámico y la sobrescritura de métodos, consolidando un criterio de diseño seguro para la inicialización de objetos. En conjunto, el desarrollo de estos ejercicios fortaleció la capacidad de estructurar clases con alta cohesión, acceso controlado y protección efectiva de la información.

7. Referencias

- [1] Universidad Nacional Autónoma de México (UNAM CUAED), “Abstracción y Encapsulamiento,” *Repositorio de la Coordinación de Universidad Abierta y Educación a Distancia*, 2024. [En línea]. Disponible en: https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3076/mod_resource/content/1/UAPA-Abstraccion-Encapsulamiento/index.html
- [2] GeeksforGeeks, “Object-Oriented Programming (OOPs) Concepts in Java,” *GeeksforGeeks Programming Series*, 2024. [En línea]. Disponible en: <https://www.geeksforgeeks.org/java/object-oriented-programming-oops-concept-in-java/>
- [3] Oracle, “Controlling Access to Members of a Class,” *The Java™ Tutorials*, 2023. [En línea]. Disponible en: <https://docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html>
- [4] CiberAula, “Encapsulamiento en Java,” *Cursos de Programación en Java*, 2023. [En línea]. Disponible en: https://www.ciberaula.com/cursos/java/encapsulamiento_java.php
- [5] GeeksforGeeks, “Encapsulation in Java,” *GeeksforGeeks Programming Series*, 2024. [En línea]. Disponible en: <https://www.geeksforgeeks.org/java/encapsulation-in-java/>
- [6] CodeGym, “Getters y setters en Java,” *CodeGym Articles*, 2023. [En línea]. Disponible en: <https://codegym.cc/es/groups/posts/es.13.getters-y-setters>